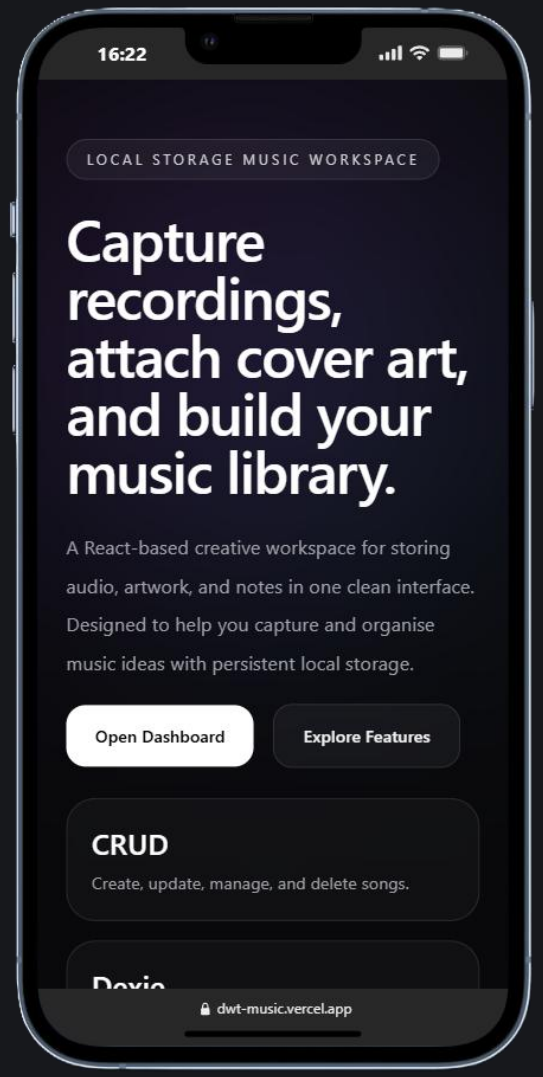
Music App

React-based audio capture and storage workspace

Student: Corey Black
Banner ID: B01651145
Module: COMP10013 Dynamic Web Technologies

Key focus

Local-first CRUD music manager using browser storage, media handling, and mobile-friendly UI design.



Landing page captured from the running Vite app

Problem, Aim, and Audience

Why this app exists and what it tries to improve

Problem

Music ideas are often split across voice memos, notes apps, and photo galleries. That makes capture slow and retrieval inconsistent.

Aim

Build a single React app where users can create tracks, attach media, save notes, and manage ideas in one workflow.

Target user

Musicians, producers, and songwriters who need fast mobile-friendly idea capture without backend complexity.

Why the local-first approach matters

- Instant save and retrieval in the browser
- No account or server dependency for the MVP
- Better privacy because files stay on-device
- Faster to deploy and easier to demonstrate live

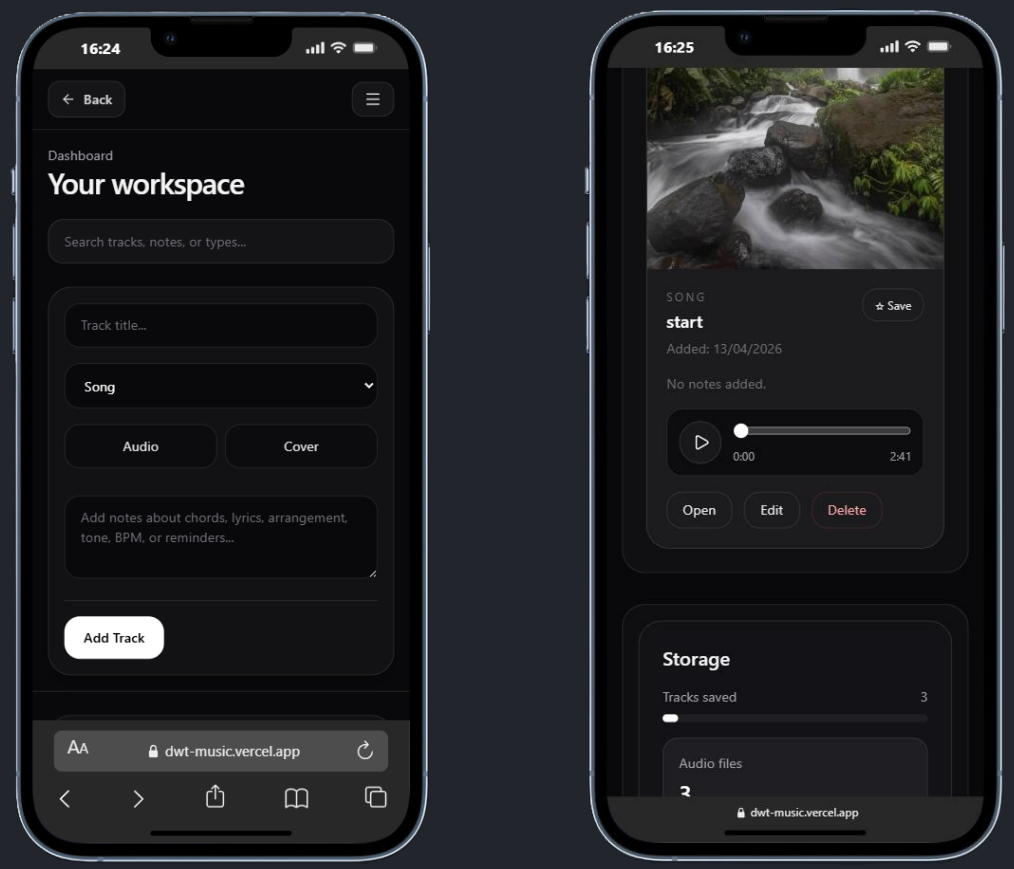
Assessment fit

This project aligns strongly with the rubric areas on mobile-focused UX, persistent on-device storage, device APIs, and code quality.

Feature Set and End-to-End User Flow

The full path from capture to retrieval

- Create tracks with title, notes, and track type
- Upload audio and preview it immediately in-app
- Choose a cover image from camera or gallery
- Edit, favourite, search, and delete saved tracks
- Open a modal view for playback and detailed management
- Persist metadata and media between sessions



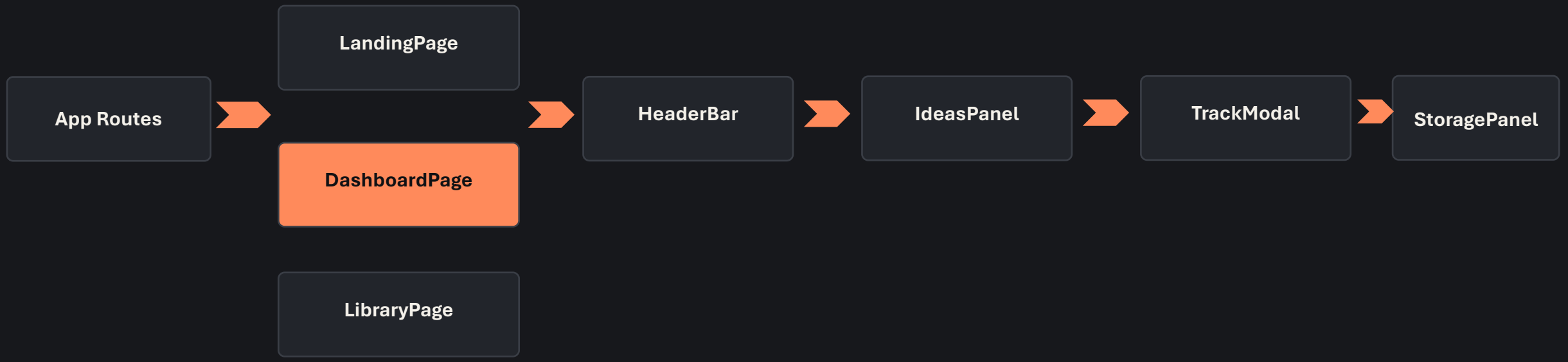
Dashboard with stored tracks, media previews, and storage summary

User journey

Landing page -> Dashboard -> Create track -> Save to browser storage -> Library -> Track modal for playback, edit, favourite, and delete

Architecture and Component Structure

Modular React design with clear page and component roles



Key architectural decision

Each page focuses on a user task, while reusable components isolate playback, media selection, storage display, and modal editing. This keeps the app easier to read, test, and extend.

Routing layer

```
<Routes>
  <Route path="/landing" element={<LandingPage />} />
  <Route path="/dashboard" element={<DashboardPage />} />
  <Route path="/library" element={<LibraryPage />} />
</Routes>
```

On-Device Storage and CRUD

CRUD functionality logic within the application per the rubric specs

localStorage

Stores lightweight metadata such as id, title, type, notes, favourite state, and timestamps.

IndexedDB with Dexie

Stores larger audio and cover image data separately using a dedicated media table and async helper functions.

Why split storage

Text data is fast to serialize in localStorage, while browser databases are better suited to richer media persistence.

CRUD coverage

- Create: save track metadata and media together
- Read: load tracks into dashboard, library, and modal views
- Update: edit metadata, replace audio, or replace cover art
- Delete: remove both the track object and related media record

localStorage hook

```
const [state, setState] = useState(() => {
  const storedValue =
localStorage.getItem(key);
  return storedValue !== null ?
JSON.parse(storedValue) : defaultValue;
});

useEffect(() => {
  localStorage.setItem(key,
JSON.stringify(state));
}, [key, state]);
```

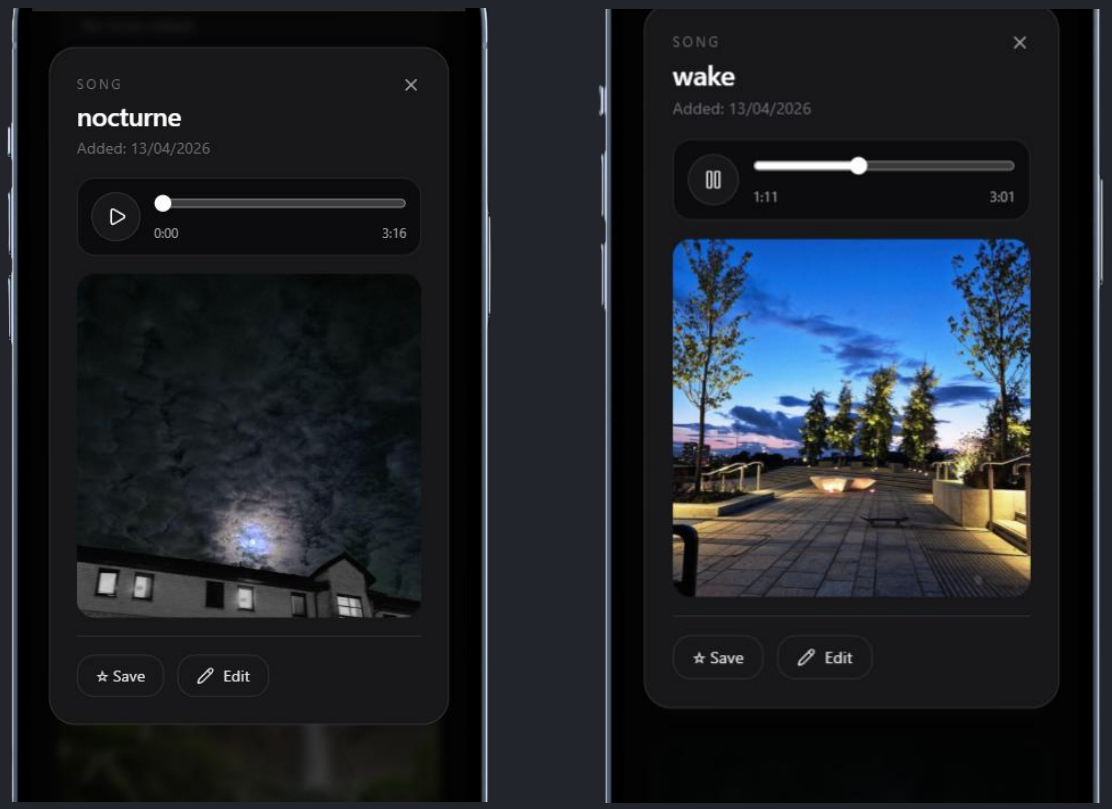
Dexie media helpers

```
export async function saveMedia(id,
mediaData = {}) {
  await db.media.put({
    id,
    audioSrc: mediaData.audioSrc || '',
    coverImg: mediaData.coverImg || '',
    updatedAt: new Date().toISOString(),
  });
}
```

Device APIs and Media Processing

Using browser capabilities to improve mobile usability

- FileReader API converts uploaded media into browser-storable data URLs
- Camera capture uses a file input with the capture attribute for mobile devices
- Canvas API crops cover art to a square format for a consistent library grid
- Notification API gives upload and save feedback to the user
- Vibration API provides tactile confirmation after successful actions



Track modal with in-app playback and media preview

Camera chooser

```
input.setAttribute('accept',
'image/*');
input.setAttribute('capture',
'environment');
input.click();
```

Image crop helper

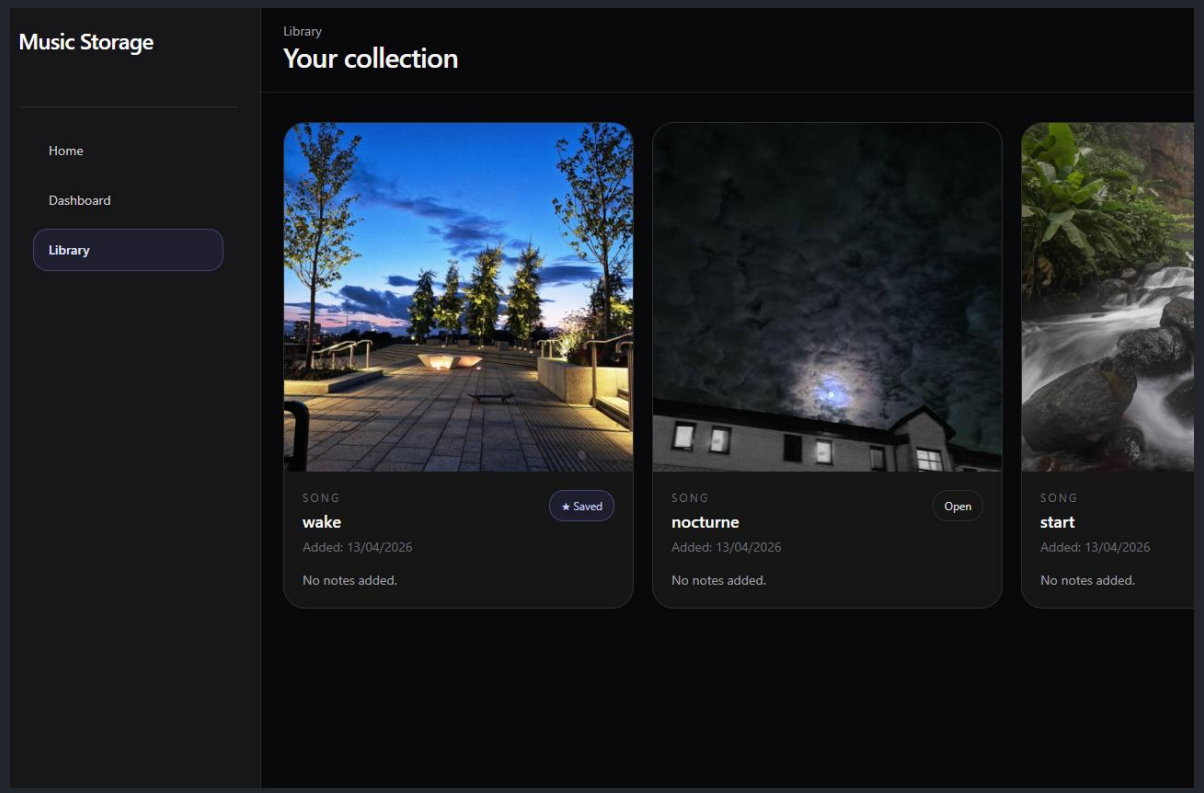
```
context.drawImage(
  image, sx, sy, cropSize, cropSize,
  0, 0, targetSize, targetSize
);
return canvas.toDataURL(mimeType, quality);
```

Feedback

```
if (Notification.permission === 'granted') {
  new Notification(title, { body });
}
if ('vibrate' in navigator) {
  navigator.vibrate(pattern);
}
```

UI and User Experience Design

Mobile-first decisions that improve speed and clarity



Library view with cover art, saved favourites, and searchable collection

Usability decisions

Responsive layout, simple navigation, large tap targets, and clear separation between creating and reviewing tracks.

- Search works across title, notes, and track type
- Track modal reduces navigation friction during review
- Audio player supports playback and seeking inside the app
- Storage panel gives quick feedback on saved content
- Dark visual theme supports media-focused browsing

Code Quality and Engineering Choices

Why the implementation is maintainable rather than just functional

Separation of concerns

State persistence, IndexedDB access, image processing, and UI rendering are split into focused modules.

Resilience

File operations and storage calls use error handling, and the app is wrapped in an ErrorBoundary.

Reusability

AudioPlayer and TrackModal centralise complex behaviour so it is not duplicated across pages.

Error boundary usage

```
return (
  <ErrorBoundary>
    <BrowserRouter>
      <Routes>...</Routes>
    </BrowserRouter>
  </ErrorBoundary>
);
```

Track update flow

```
setTracks((prev) =>
  prev.map((track) =>
    track.id === trackId
      ? { ...track, title, notes, updatedAt:
now }
      : track
  )
);
```

Audio player state

```
const [isPlaying, setIsPlaying] = useState(false);
const [currentTime, setCurrentTime] = useState(0);
const [duration, setDuration] = useState(0);
```

These choices support the rubric area on clean, readable, well-structured React code and show clear reasoning behind the architecture.

Deployment and Demonstration Plan

How the application is run and how it will be shown live

Production deployment

```
Platform: Vercel
Live URL: https://YOUR-VERCEL-URL.vercel.app
Build command: npm run build
Output directory: dist
```

Deployment status

The app is publicly deployed on Vercel, so assessors can access the live version without local setup.

Live demo order

Landing page, dashboard, create track, save media, open library, search, playback, edit, favourite, and delete.

Demonstration

- Open live Vercel URL at the start of demonstration
- Showcase a complete CRUD cycle
- Explain where metadata and media are stored while saving track
- Use the storage panel and modal to reinforce persistence and UX quality

Demo focus

Show the app is working completely, explain design choices, features are in line with rubric: UI/UX, APIs, structured project

Critical Reflection

Challenges, tradeoffs, and lessons learned

Challenges

Managing media inside browser storage, keeping UI interactions simple, and coordinating localStorage with IndexedDB without a backend.

Tradeoffs

Using data URLs simplified implementation, but it is less storage-efficient than using Blob objects or remote storage.

Lessons learned

React state and browser storage can support a solid MVP when responsibilities are clearly separated.

- Local-first design was the right scope for an MVP and coursework timeline
- Browser APIs can make a web app feel much closer to a mobile app
- Architecture decisions matter more than simply listing libraries
- Future work should improve storage efficiency, documentation, and extensibility

Future Improvements and Conclusion

What would move this from MVP to a stronger product

- Add cloud sync for multi-device access
- Store audio and images more efficiently with Blob-based handling
- Introduce waveform visualisation and audio trimming
- Expand tagging, sorting, and smarter search
- Improve README and setup guidance for assessors and other developers
- Optionally add PWA installability and offline caching

Conclusion

This music-app is a working React project that demonstrates persistent CRUD storage, browser device APIs, responsive UI design, and modular component architecture. It is already a complete local-first MVP and provides a strong foundation for future expansion.

Thank you